

Supplementary Materials: Prediction-Driven Non-Preemptive Scheduling with Bounded Overtaking for Shared Execution Services under Deadline-Driven Bursty Load

[First Author Name]¹ and [Second Author Name]^{2,*}

This file holds supporting material for the manuscript: the visibility audit of the feature pipeline and the predictor scores on the four non-judging traces (Section S1); the field audit of the main judging trace and two checks on the continuous-integration refit (Section S2); five tables of experimental detail, the selected budget points, the two budget shapes the main guard table no longer prints, the identity residual on all nine cells and the single-server configuration (Section S3); and two deferred proofs, of the workload comparison and of the consistency proposition (Section S4). Section numbers here carry an S prefix and every table is numbered S. A cross-reference without an S — “Section 8.2”, “Table 8”, “Theorem 1” — points into the main manuscript, since a \ref across the two files does not resolve. The traces are the ones the manuscript’s Table 3 names: the CodeBench judging logs [1], the Azure Functions invocation trace [2], the Intel Netbatch logs [3] and the two Firefox continuous-integration pools collected from Mozilla’s public APIs [4,5]. The additional reproducibility specification is Section S5; capacity, physical queueing, feedback and LPC-EGEE results are Section S6. Additional theory is Section S7. Every empirical figure quoted here is a development figure in the sense of Section 8.2 of the manuscript: nothing in this file comes from a sealed part of any trace, and nothing here was recomputed for it.

S1. Cost Prediction

S1.1. The Visibility Audit in Full

Section 4.2 of the manuscript states that every historical feature is recomputed by streaming the log in availability order and reading each entity’s state as of the query instant, and that 0 rows disagree in any of the eight configurations. This is what that test is.

On 3,000 randomly drawn development jobs all 34 history and relational features are recomputed from scratch under each configuration’s own visibility rule and compared against the pipeline’s values. The same test applied to the natural but wrong construction, in which a feature may use any record with a smaller index, fails on 97.67% of rows under the primary rule, so the test does discriminate. The graph pipeline is checked separately: an index assertion over 863,149 rows finds 0 timeline entries pointing at or after the read, brute-force recomputation on 600 rows matches on all 2,400 root states and all 2,400 neighbour sets, and perturbing or deleting every record after a cut time leaves all earlier predictions unchanged while a positive control moves the later ones.

S1.2. Predictors on the Non-Judging Traces

Table S1 is the cross-trace counterpart of Table 1 of the manuscript. The three figures Section 4.4 quotes from it are the serverless STATIC Spearman correlation and the two CI pools’ ENT AUROC; the AUROC column of Table 11 of the manuscript carries the figure the scheduling results there are read against, which is the ALL row of each trace.

Table S1. The same protocol on the non-judging traces, by feature group. ENT uses the entity's own visible history, STATIC uses only what is declared before the job runs, ALL is both. RMSE and Spearman are on $\log(1 + C)$.

Trace	Features	AUROC	AUPRC	RMSE	Spearman
Serverless invocations	ENT	0.9828	0.816	0.3401	0.953
	STATIC	0.6238	0.084	0.8088	0.021
	ALL	0.9847	0.830	0.3185	0.932
CI pool, hardware A	ENT	0.6334	0.157	0.3964	0.833
	STATIC	0.4990	0.045	0.7659	0.277
	ALL	0.6512	0.177	0.3956	0.837
CI pool, hardware B	ENT	0.9968	0.984	0.0887	0.985
	STATIC	0.6915	0.077	0.6411	0.500
	ALL	0.9970	0.985	0.0860	0.988
Compute farm	ENT	0.8903	0.265	1.4214	0.883
	STATIC	0.8704	0.242	2.2559	0.750
	ALL	0.8902	0.260	1.3485	0.892

S2. Traces and Workload Characterisation

S2.1. The Field Audit of the Main Judging Trace

Section 7.2 of the manuscript states four readings of the main trace. This is the evidence for them, and the two archive defects that follow from reading the trace this way.

The timestamp is a result stamp.

Among later-period runs the gap to the same user's previous event grows one for one with the cost (slope 0.99 of the median gap on C) while the gap to the next event barely moves (slope 0.16), and the feedback after a user kills a run arrives within about 0.1 s whatever its length. A result stamp predicts both readings; a receipt stamp neither.

The whole-second leak.

Stamps are recorded to the whole second while costs are recorded to the microsecond, so the fractional part of a job's cost appears in the gap between its own stamp and the next event, and a feature like "seconds since the user's previous record" reads the label off the clock. We close it by replacing t with $t + u$, where $u \in [0, 1)$ is drawn deterministically from a fixed key and the record's identifier.

The cost field and the limit.

The cost field is wall-clock occupancy of one evaluation including interpreter start-up, read from the 10th percentile of correct submissions (0.12–0.18 s across the two periods) and from its weak dependence on the number of test cases (median $0.196 + 0.007n$ s in the later period). The platform limit is 60 s: the last run to reach it ends at 2019-08-28 10:32, after which the largest observed cost is 59.97 s, with 13 runs in $[59, 60)$ s and none above. Before that date there was no fixed limit, the longest run is 8,901 s, and the 1,189 runs that end with a kill are almost all above 360.8 s, pointing at a container watchdog rather than judging demand.

Co-ending groups, zero-cost blocks and two short archives.

Groups of runs by the same user that end in the same second are 7.53% of blocks and 26.28% of capped work; every prediction table in the manuscript is repeated on rows outside such groups. A further 1,671 blocks record a cost of exactly zero with empty outputs; they leave the simulated trace and stay in the features, the labels and every total. The 2019-2 archive stops at 2019-09-19, months before that term's assessments end, and 2020-1 is short by about 3% of its blocks; both serve per-submission modelling and are excluded from anything that treats a term as a whole.

Against the published release statistics.

We match the class count in 18 of 18 terms and the student and exercise counts in 16 of 18, and parse 4.08% fewer execution blocks than the official column sums, whose printed totals disagree with those same columns by -248 exercises and $+246,246$ blocks. We quote the columns.

S2.2. The Continuous-Integration Refit: Sharpness and Circularity

Section 7.3 of the manuscript reports the held-out refit of the effective capacity k_{eff} and the setup term on the two Firefox CI pools. Two checks on that fit sit here.

The objective is sharp in k rather than flat: on a step-of-one grid only 5 to 8 values of k put the simulated mean wait within 10% of the recorded one, so the capacity the fit chooses is not one of many equally good answers.

The circularity of the original whole-window fit can be read off the two pools directly. On pool A the whole-window and held-out rows are almost identical, because every split picks nearly the same capacity; on pool B the reverse split’s held-out capacity gives a mean ratio of 1.14 on its test part where the whole-window capacity gives 0.99. The difference between those two numbers is the size of the circularity that fitting on the reported window would have hidden.

S3. Experiments

S3.1. What the Selection Rule Chose

Table 5 of the manuscript gives the three pre-stated grids, the feasibility rule, the objective and the tie-break, and names the shape the rule selected at each promise. Table S2 gives the selected points in full and the nine per-shape winners. The same values are in `protocol_lock.draft.json`, which is the artefact the selection was frozen into.

Table S2. The three selected budget points and the nine per-shape grid winners. Budgets in work scale as $k/4$; the harm limit is $G/2$, the feasibility threshold of the selection rule. “Feasible” counts the grid points that met it in all 15 validation cells.

Item	Value
Selected, $G = 300$ s	age-relative: $B_0 = 60k/4$ s, $\eta = 0.50$, $\gamma = 0$; worst-cell gap 0.4251, worst-cell harm 122.8 s against a 150 s limit, 44 of 99 candidate points feasible
Selected, $G = 600$ s	age-relative: $B_0 = 120k/4$ s, $\eta = 0.75$, $\gamma = 0$; worst-cell gap 0.7216, worst-cell harm 293.3 s against 300 s, 70 of 107 feasible
Selected, $G = 1200$ s	queue-length: $B_0 = 0$, $\eta = 0$, $\gamma = 16k/4$ s; worst-cell gap 0.7283, worst-cell harm 145.3 s against 600 s, 95 of 111 feasible
Per-shape winners	constant: $B_0 = 329.671, 981.126, 2,029.958 k/4$ s at the three promises; age-relative: $(60, 0.50), (120, 0.75), (360, 0.75)$; queue-length: $\gamma = 16k/4$ s with $B_0 = 0, 120k/4$ s, 0

S3.2. The Two Budget Shapes the Main Table No Longer Prints

Section 8.4 of the manuscript compares the three budget shapes at a fixed promise through the paired differences, which is how the comparison is read. Table S3 gives the age-relative and queue-length rows themselves, in the format of Table 8 and in the same cells; the Guard(G), Guard-fixed(G), Fixed(G) and Skip(G) rows stay there. One of the three shape winners coincides with Guard(G) at each promise, which is why some rows below repeat a row of Table 8.

Table S3. The age-relative and queue-length winners at three promises and three loads, in the format of Table 8 of the manuscript. Excess and harm are the worst over the five overlays, the rest are means; budgets scale as $k/4$.

Load	Policy	Promise	p99 _{dl}	Gap closed	Red. (%)	Max exc.	Harm	Fired (%)
$\rho = 0.5, k = 7.8$								
	Guard-age(300)	300	15.58	0.705 [0.678, 0.724]	43.7	192.5	132.1	1.35
	Guard-queue(300)	300	15.27	0.723 [0.697, 0.742]	44.8	193.4	94.3	21.60
	Guard-age(600)	600	14.97	0.740 [0.713, 0.760]	45.8	492.2	237.0	0.17
	Guard-queue(600)	600	14.88	0.744 [0.718, 0.765]	46.1	504.5	106.0	0.16
	Guard-age(1200)	1200	14.73	0.751 [0.723, 0.773]	46.5	1,049.1	237.0	0.01
	Guard-queue(1200)	1200	14.84	0.746 [0.719, 0.768]	46.2	907.8	94.3	22.01
$\rho = 0.8, k = 5$								
	Guard-age(300)	300	55.94	0.738 [0.697, 0.761]	53.0	209.7	114.1	11.57
	Guard-queue(300)	300	54.69	0.753 [0.713, 0.777]	54.0	210.7	117.6	17.53
	Guard-age(600)	600	47.47	0.837 [0.815, 0.851]	60.1	507.8	319.6	1.31
	Guard-queue(600)	600	47.69	0.834 [0.814, 0.847]	59.9	508.5	117.7	1.35
	Guard-age(1200)	1200	45.16	0.864 [0.844, 0.876]	62.0	1,102.9	370.5	0.29
	Guard-queue(1200)	1200	46.85	0.844 [0.823, 0.857]	60.6	1,104.9	117.6	9.93
$\rho = 1.0, k = 4$								
	Guard-age(300)	300	163.40	0.478 [0.387, 0.602]	40.2	229.1	134.8	28.64
	Guard-queue(300)	300	169.22	0.453 [0.368, 0.607]	38.1	231.4	144.8	30.99
	Guard-age(600)	600	89.24	0.801 [0.764, 0.842]	67.3	523.7	339.9	7.15
	Guard-queue(600)	600	91.09	0.793 [0.758, 0.833]	66.6	515.8	123.4	7.08
	Guard-age(1200)	1200	72.52	0.874 [0.851, 0.893]	73.4	1,117.3	453.3	0.81
	Guard-queue(1200)	1200	82.67	0.830 [0.801, 0.857]	69.7	1,110.4	144.8	6.21

S3.3. The Residual on All Nine Cells

Table 10 of the manuscript prints the three Guard(600) cells and one reversed score. Table S4 gives all nine. The reading Section 8.5 takes from them is that the residual is a property of the schedule and barely moves with the policy: inside each load the ratios are similar in scale, but are not equal to the third decimal. An R^2 rounded to unity is likewise not exact equality.

Table S4. The residual $D_i = k(W_P[i] - W_{FCFS}[i]) - (\text{In}_i - \text{Out}_i)$ of Theorem 1 of the manuscript on overlay 0, 17,634,760 jobs per cell. Ratio is $\max |D_i|$ over the bound $2(k-1)L$, which is $14L$, $8L$ and $6L$ at the three loads; R^2 and the worst error are for $(\text{In}_i - \text{Out}_i)/k$ against the realised excess over first-come first-served.

Load	Policy	$\max D_i /L$	Ratio	$D_i = 0$	R^2	worst err. (s)
$\rho = 0.5, k = 8$	SPJF-E	6.788	0.485	95.6%	0.985	50.9
	Guard(600)	6.788	0.485	95.6%	0.983	50.9
	Reversed score	6.788	0.485	95.6%	0.996	50.9
$\rho = 0.8, k = 5$	SPJF-E	3.922	0.490	87.2%	0.996	47.1
	Guard(600)	3.922	0.490	87.2%	0.995	47.1
	Reversed score	3.881	0.485	87.5%	0.999	46.6
$\rho = 1.0, k = 4$	SPJF-E	3.082	0.514	81.1%	0.998	46.2
	Guard(600)	3.082	0.514	81.1%	0.998	46.2
	Reversed score	2.994	0.499	81.7%	1.000	44.9

S3.4. A Single Server

Section 8.6 of the manuscript reports the single-server configuration through five gap figures, three harm ratios and two realised excesses. Table S5 is the configuration in full. The parameters are the ones the selection rule chose on the 15 multi-server validation cells; $k = 1$ is not among them, and we do not re-tune on it.

The firing rate of the three unguarded policies is printed ---: the column counts dispatch epochs at which the guard overrides the base policy, and these three runs have no

guard. Scoring first-come first-served at 100% by the convention that every first-come first-served dispatch serves the queue head, as an earlier version of this table did, compares it with the guarded runs on a quantity it does not have.

Table S5. Single server, busy-hour $\rho = 0.80$, 3,209,347 jobs. Waits and excesses in seconds. The parameters are the ones the rule selected on the multi-server validation cells; this configuration is not among them. The skip count is $\kappa = \lfloor Gk/L \rfloor - (2k - 2)$.

Policy	Promise	p99 _{dl}	Gap closed	Max exc.	Harm	Fired (%)
FCFS	—	221.27	0.000	0.0	0.0	—
SJF on true sizes	—	45.32	1.000	2,054.5	777.9	—
SPJF-E	—	60.91	0.911 [0.854, 0.933]	2,520.3	656.4	—
Guard(300)	300	89.96	0.746 [0.571, 0.829]	297.9	81.8	8.17
Guard-fixed(300)	300	125.68	0.543 [0.207, 0.718]	140.3	119.5	17.09
Guard-age(300)	300	89.96	0.746 [0.571, 0.829]	297.9	81.8	8.17
Guard-queue(300)	300	105.15	0.660 [0.532, 0.715]	297.9	53.9	29.15
Fixed(300)	300	83.18	0.785 [0.584, 0.865]	297.9	246.3	6.33
Skip(300), $\kappa = 5$	300	220.39	0.005 [-0.002, 0.019]	95.1	81.8	73.17
Guard(600)	600	71.07	0.854 [0.780, 0.895]	595.5	165.5	2.53
Guard-fixed(600)	600	82.10	0.791 [0.602, 0.868]	303.0	246.3	6.16
Guard-age(600)	600	71.07	0.854 [0.780, 0.895]	595.5	165.5	2.53
Guard-queue(600)	600	91.52	0.737 [0.640, 0.812]	582.0	81.8	3.43
Fixed(600)	600	68.48	0.868 [0.783, 0.906]	593.4	563.6	2.29
Skip(600), $\kappa = 10$	600	219.89	0.008 [-0.004, 0.037]	100.9	83.2	66.98
Guard(1200)	1200	98.00	0.701 [0.640, 0.742]	1,157.4	53.9	26.15
Guard-fixed(1200)	1200	69.03	0.865 [0.789, 0.905]	564.2	509.3	2.57
Guard-age(1200)	1200	63.80	0.895 [0.835, 0.920]	1,182.3	366.4	0.68
Guard-queue(1200)	1200	98.00	0.701 [0.640, 0.742]	1,157.4	53.9	26.15
Fixed(1200)	1200	63.19	0.898 [0.837, 0.923]	1,180.4	656.4	0.45
Skip(1200), $\kappa = 20$	1200	217.64	0.021 [-0.008, 0.084]	168.0	97.7	58.67

S4. Deferred Proofs

Appendices A.1 and A.5 of the manuscript give the ideas behind the arguments proved here. The workload comparison uses continuity and left limits, whereas consistency is proved by induction. The tightness constructions remain in Appendices A.2–A.4, A.6 and A.7 of the manuscript. We use the manuscript's notation: $U_Q(t)$ is the unfinished work at time t under policy Q , and $N_Q(t)$ is the number of jobs present.

S4.1. The Workload Comparison

Proof of Lemma 1 of the manuscript. Put $D = U_{Q_1} - U_{Q_2}$. Arrivals add the same jump to both terms, so D is continuous, and between arrivals it changes at rate $-(\min(N_{Q_1}, k) - \min(N_{Q_2}, k))$; it is therefore piecewise linear with breakpoints among the finitely many arrival and completion instants of the two runs.

If $U_{Q_1}(u) > (k - 1)L$ then more than $k - 1$ jobs are present under Q_1 , each carrying at most L , so every server of Q_1 is busy and $\min(N_{Q_1}, k) = k \geq \min(N_{Q_2}, k)$: D does not increase at u . Fix t and let $S = \{u \leq t : U_{Q_1}(u) \leq (k - 1)L\}$, which is non-empty because U_{Q_1} vanishes before the first arrival, and put $t_0 = \sup S$. The supremum need not be attained, since U_{Q_1} jumps upwards at arrivals, so we use the left limit: there are $u_n \uparrow t_0$ with $U_{Q_1}(u_n) \leq (k - 1)L$, hence $U_{Q_1}(t_0^-) \leq (k - 1)L$ and, D being continuous, $D(t_0^-) \leq U_{Q_1}(t_0^-) \leq (k - 1)L$ because $U_{Q_2} \geq 0$. On $(t_0, t]$ we have $U_{Q_1} > (k - 1)L$, so D does not increase there, and $D(t) \leq (k - 1)L$. Exchanging Q_1 and Q_2 gives the other direction. That the constant cannot be lowered is the cascade of Appendix A.2 of the manuscript, which reaches $(k - 1)L(1 - ((k - 1)/k)^m)$ after m rounds. At $k = 1$ the bound reads $|D| \leq 0$ and is attained everywhere: one server executes work at rate 1 whenever any job is present, so $U_{Q_1} \equiv U_{Q_2}$. $\square$

Proof of the strict inequality in Lemma 1, for $k \geq 2$. Keep the notation above, so $D = U_{Q_1} - U_{Q_2}$ is continuous and piecewise linear with finitely many pieces. Suppose $D(t) = (k-1)L$ for some t and let t_1 be the smallest such t ; it exists because $\{D = (k-1)L\}$ is closed and $D(0) = 0 < (k-1)L$, which uses $L > 0$ and $k \geq 2$.

Let I be the linear piece immediately to the left of t_1 , which has positive length. Its slope is strictly positive: a negative slope would put $D > (k-1)L$ somewhere on I and a zero slope would put $D = (k-1)L$ on all of I , and either contradicts the minimality of t_1 . The slope of D is $-(\min(N_{Q_1}, k) - \min(N_{Q_2}, k))$, so $\min(N_{Q_1}, k) < \min(N_{Q_2}, k)$ on I and therefore $N_{Q_1} \leq k-1$ there.

With at most $k-1$ jobs present and Q_1 work-conserving, none of Q_1 's jobs waits, so all of them are in service; hence U_{Q_1} falls at rate N_{Q_1} on I , and $U_{Q_1} \leq (k-1)L$ on I because each of the at most $k-1$ present jobs carries at most L . Taking left limits at t_1 and using the continuity of D ,

$$U_{Q_1}(t_1^-) - U_{Q_2}(t_1^-) = (k-1)L, \quad U_{Q_1}(t_1^-) \leq (k-1)L, \quad U_{Q_2} \geq 0,$$

which forces $U_{Q_2}(t_1^-) = 0$ and $U_{Q_1}(t_1^-) = (k-1)L$. Since $L > 0$ and $k \geq 2$, this gives $N_{Q_1} \geq 1$ on I — with $N_{Q_1} = 0$ we would have $U_{Q_1}(t_1^-) = 0 = (k-1)L$ — so U_{Q_1} is strictly decreasing on I and $U_{Q_1}(t_1^-) < \sup_I U_{Q_1} \leq (k-1)L$, a contradiction. Exchanging Q_1 and Q_2 gives the other direction. Both steps that invoke $(k-1)L > 0$ fail at $k = 1$, where the statement is false: there $U_{Q_1} \equiv U_{Q_2}$ and the bound 0 is attained at every instant. $\square$

S4.2. Consistency, and Why the Converse Fails

Proof of Proposition 1 of the manuscript. Induct on dispatch epochs: up to the first epoch at which the two runs could differ their completion histories agree, hence so does $\text{over}[\cdot]$, so $E(t)$ is empty in the wrapped run and it copies A . The first consequence uses $\text{over}[q](t) \leq \text{In}_q^A$, a completed overtaker of q having been dispatched while q waited; the second uses Corollary 2 of the manuscript. $\square$

The converse fails, so this is not an equivalence. Take $k = 1$, two jobs of service 1 at $t = 0$, the base policy “higher rank first”, and $B = 1$. Both runs dispatch in the order $(1, 0)$, yet at the second dispatch the waiting job has $\text{over} = 1 \geq B$, so the guard fires and happens to choose what the base would have chosen. Nor is there multiplicative consistency: with $k = 1$, one job of service L followed by m jobs of service 1 all at $t = 0$, the wrapper with $B = 0$ is first-come first-served while the base ordering by service time is not, and the ratio of mean waits is $(2L + m - 1)/(m + 1)$, unbounded in L .

S5. Reproducibility and Provenance of the Empirical Results

S5.1. Feature Definitions, Targets and Rolling Splits

The executable feature dictionary is `src/spjf_guard/features/sweep.py`: `HIST_COLS`, `CTX_COLS`, `GROUPS` and `feature_frame` specify the entity aggregates, missing values and event order; `src/spjf_guard/data/events.py` specifies code columns and clock inputs. M4 combines static code, assessment/calendar context, exercise history, user history, user-exercise history and previous-event history. It excludes the relational extensions. Outcome aggregates contain log-cost moments and quantiles, error/heavy rates and recent-result summaries; counts and elapsed time since arrival are known at arrival and are distinguished from outcome fields. The rolling-history window, tie order and update formulas are defined in that dictionary. Static uses only the code and context blocks. The independent source-clock audit in Section S1 verifies that dictionary under its original clock, not under an arbitrary replayed policy.

Package scores are fitted by `src/spjf_guard/predict/forward.py`. For each target term, training includes only terms whose first arrival precedes that target, and only records whose result is readable strictly before its first arrival. History thresholds and user-slowness cutpoints are refitted from those training rows; a model is then frozen for the target term. The targets are 2020-ERE, 2020-2, 2021-1, 2021-2, 2022-1 and 2022-2. Starting at 2018-1, their training periods end respectively at 2019-2, 2020-1, 2020-2, 2021-1, 2021-2 and 2022-1; availability filtering still applies at each cutoff. Validation overlays exclude the final development term from selection, but that term and the earlier family exploration are development data, not sealed testing.

The score objectives are in `src/spjf_guard/predict/scores.py`, and the deterministic fit settings in `lgb_parameters` and the configuration identified by the output manifest. `spjf_e` is a Tweedie regression of capped seconds; `spjf_log` is squared-error regression of log-transformed cost. In contrast, the earlier M4 AUROC 0.9283 [0.9118, 0.9415] is the heavy-class classifier probability, averaged over 3 seeds, trained on 2018-1–2019-2 and tested only on 2022-2. Those retrospective classifier predictions never enter scheduling. The package's pooled `spjf_e` AUROC 0.9360 combines 402,358 unreplicated forward-prediction rows. Its evaluation heavy threshold stays at the core training p95, 1.559043 s; it is not the threshold refitted inside each model's history features. Predictor intervals resample users, whereas scheduling intervals resample weeks. The two AUROCs compare different fits, targets and pooling and must not be read as a measured improvement on a common split.

S5.2. Exact Selection Grids and Counts

The full expanded grid, its definitions, counts and deduplication totals are in `outputs/selection_v3/selection_protocol.json`; per-cell values, worst-cell values and selections are in the adjacent `selection_grid.csv`, `selection_worst.csv`, `selection_frontier.csv` and `selected_parameters.csv`. All budget and queue-work bases below scale by $k/4$ before dispatch arithmetic. The cap is determined by the promise.

The constant grid has the following 39 base budgets (seconds), shared across promises:

15, 17.99, 21.576, 25.877, 30, 31.035, 37.222, 44.641, 53.54, 60, 64.212, 77.012, 92.363, 110.774, 120, 132.855, 159.338, 191.099, 229.192, 240, 274.878, 329.671, 360, 395.386, 474.201, 480, 568.726, 600, 682.093, 818.058, 981.126, 1176.7, 1411.26, 1692.57, 2029.96, 2434.6, 2919.9, 3501.94, 4200.

It is the union of 32 logarithmically spaced values between 15 and 4200 s, rounded as recorded, and the anchors 15, 30, 60, 120, 240, 360, 480, 600, with duplicate values removed. A full-cap constant point is added at each of $G = 300, 600, 1200$ s. This makes 42 expanded constant entries.

For each promise the age-relative grid is the Cartesian product

$$B_{0,\text{base}} \in \{0, 15, 30, 60, 120, 240, 360, 480, 600\}, \quad \eta \in \{0, 0.25, 0.5, 0.75, 0.9, 0.95\}, \quad \gamma = 0.$$

The queue-length grid is

$$B_{0,\text{base}} \in \{0, 30, 120\}, \quad \eta \in \{0, 0.9\}, \quad \gamma_{\text{base}} \in \{1, 4, 16\}.$$

Thus the family counts over all promises are $42 + 162 + 54 = 258$ expanded points. Deduplicating identical dispatch rules after scaling and saturation leaves 243 distinct schedules at each evaluated server count. This is a computational deduplication, not a reduction of the declared search space.

Candidate counts per promise answer a different question. The constant entries shared across promises qualify for a given G only if their worst-cell certified promise is at most G ; an age-relative or queue-length entry belongs to its own declared promise. Feasibility then additionally requires $\text{harm} \leq G/2$ in every validation cell. Table S6 separates these filters, explaining why the per-promise totals differ from the expanded search size. The objective is the minimum deadline gap over all validation cells, with ties resolved by smaller worst harm, then smaller B_0 , η and γ .

Table S6. Candidates / feasible points after promise assignment and the harm filter.

G (s)	Constant	Age-relative	Queue-length	Joint
300	27 / 22	54 / 15	18 / 7	99 / 44
600	35 / 31	54 / 30	18 / 9	107 / 70
1200	39 / 35	54 / 43	18 / 17	111 / 95

The aging comparator has the credit grid 0.00001, 0.00003, 0.0001, 0.0003, 0.001, 0.003, 0.01, 0.03, 0.1, 0.3 and 1. It uses the same validation pool, harm threshold and worst-cell objective, with ties to less harm then larger credit. It uses conservative scores; the guard grids used the original package score. The selected guard parameters are subsequently held fixed across visibility controls, so this is not a claim of reselection under conservative or exact histories. The aging label carries no proved guarantee.

S5.3. Replication, History and Aggregation Order

Each overlay is built from the same 40 class-terms replicated 44 times with independent whole-week shifts; each of the five overlays therefore reuses the same jobs. Original scores are computed on the source calendar and copied with their source jobs. Conservative histories are namespaced within a class-term, use only replayed submissions and share the copy’s shift. These copies are not independently trained predictors or independent workload samples.

For the pending exact refinement, a copy means the same class-term, replication round and shift. Its own submission outcomes follow policy-specific completions. Other classes and terms are exogenous history in that copy’s original relative clock: when the target copy shifts by s_i , an external result is interpreted at $\text{done}_j + s_i$, not at another copy’s independent shift. This preserves earlier completed history without claiming a single platform whose copied users share dynamically changing state. A separate sensitivity removes entire records from other replayed classes in the same term, including arrival counts and elapsed time since arrival; it then repeats same-copy refinement. Ordinary outcome withholding preserves already-observed arrivals. The event order releases positive duration completions before reads at the same instant, and zero-duration outcomes after those reads. These conventions are specified in docs/policy_consistent_visibility.md and implemented by src/spjf_guard/features/refine.py.

In the package main and visibility tables, each overlay’s deadline p99 is first computed over its eligible jobs; its gap is then computed against that overlay’s FCFS and SJF. Reported p99 and gap are separate arithmetic means over overlays. Consequently the displayed mean waits need not algebraically recover the displayed mean gap. Harm and maximum excess are maxima over overlays. Firing is queue-weighted within a cell, then averaged across overlays. Each bootstrap draw uses the same week multiplicities for every policy and overlay, recomputes weighted job quantiles and cell-level ratios, and averages the ratios. No table uses an average of weekly p99 values. The capacity-sweep gap instead uses the ratio of averaged overlay quantiles, as explicitly identified in Section S6.

S5.4. Which Results the Package Reproduces

The package produces the main CodeBench ranking, guard, adversarial and residual tables from `outputs/dev_tables/`, including its separate single-server directory; predictor metrics from `outputs/dev_predictor/`; and visibility comparisons from `outputs/dev_visibility/`. Guard selection and aging selection are in `outputs/selection_v3/` and `outputs/selection_aging/`. Their manifests identify inputs, settings and output hashes. The preserved original recipe is `configs/main_original_84932d9.yaml`; the original outputs are not claimed to have been regenerated under an unfinished visibility revision.

Earlier predictor-family comparisons and source-clock feature sensitivities are archived under `evidence/codebench_service_v2/` and `evidence/predictor_neural/`. The refit-control and GRU rows in the main ranking table come from `evidence/main_v3/`; the ranking-loss table uses its own fits in `evidence/ranking_score/`. Cross-domain rows combine the package CodeBench row with `evidence/cross_domain/`, `evidence/accoding_v2/`, `evidence/firefox_ci/` and the LPC-EGEE study below. CI chronological-refit validation is in `evidence/firefox_ci_holdout/`. The capacity and physical experiments are separate analyses in `prechecks/weakness1_attack/`.

The trace-construction sensitivity table (Table 12 in the unrevised numbering) uses the earlier source-clock M4 log-scale score, overlays 0–2, and a fixed completed-work budget $B = 600$ s. This is neither a 600-second promise nor the selected age-relative Guard(600). It supports the earlier ordering result's sensitivity to term inclusion, the alternate timestamp interpretation, and merging or dropping co-ending runs. It does not test the final expected-cost guard, conservative history or policy-specific refinement under those altered inputs. Numerical agreement of the exact-arithmetic package and its independent reference is also distinct from the archived earlier-pipeline validation.

The sealed procedure is specified by `docs/sealed_run_procedure.md`. It produces separate multi-server, single-server, predictor and visibility outputs, including the policy-specific exact stage, with unchanged declared selection rules and parameters. It does not extend the earlier predictor-family or cross-domain studies. No sealed result is present in this supplement.

S6. Capacity, Physical Queueing and Feedback

These development experiments address distinct questions. The capacity sweep changes the size of the constructed pool while preserving its complete offered stream. The physical experiment checks an executing timed-work service on a selected window. Neither identifies a historical queue on CodeBench or a causal deployment effect. The scores are the archived embedded Tweedie predictions, not the later package refit, conservative score or pending exact visibility variant.

S6.1. Capacity Sensitivity on Complete Traces

The declared common grid uses every integer $k = 1, \dots, 20$ on all five development overlays; overlay 0 alone continues through $k = 76$. This gives 156 capacity cells. Each replay starts at the beginning of its complete trace, keeps queue state across weeks and drains all jobs; there is no time compression, window reset or thinning. The same source jobs occur in all overlays, so these are alternative constructions rather than independent deployments. The grid and comparison family were declared after a pilot on the first overlay, before the full sweep, not before inspecting the development demand.

Table S7 reports the average of the overlay-specific deadline p99 values, with point-wise intervals. Gap closure in this study is the ratio of differences of those averaged quantiles. This differs from the package main tables, which average overlay-specific gap ratios. Paired calendar-week resampling uses 2,000 common draws, recomputing quantiles

from the weighted job distribution, not averaging weekly quantiles. The declared simultaneous band covers 80 absolute FCFS-minus-policy differences: each capacity and each non-oracle policy. It uses a centred coordinate-standardised maximum statistic. These intervals resample realised outcomes, not queue trajectories; they omit source-term selection, predictor fitting, shared source jobs, model mismatch, feedback and cross-week state dependence. Maxima below are trace maxima, not population confidence bounds.

Table S7. Capacity sweep with archived scores: mean overlay deadline p99 in seconds, with point-wise paired calendar-week intervals. Negative effects are retained.

<i>k</i>	FCFS	SPJF-E	Guard(600)
1	108,345.31 [92,267.38, 114,125.36]	130,231.12 [65,565.87, 155,632.22]	108,839.10 [92,739.31, 114,582.34]
2	10,362.62 [7,565.14, 11,503.45]	1,844.21 [766.70, 3,990.37]	10,618.06 [7,820.32, 11,755.23]
3	1,056.40 [688.91, 1,273.01]	138.86 [105.79, 169.05]	934.68 [452.24, 1,253.73]
4	273.50 [197.09, 352.10]	62.92 [53.69, 72.13]	89.61 [65.94, 114.14]
5	119.35 [92.91, 143.43]	43.39 [37.94, 47.54]	47.44 [40.46, 53.39]
8	24.43 [18.46, 29.58]	13.21 [9.58, 15.80]	13.36 [9.67, 16.05]
14	0.84 [0.45, 1.50]	0.47 [0.30, 0.73]	0.47 [0.30, 0.73]
20	0.00 [0.00, 0.04]	0.00 [0.00, 0.03]	0.00 [0.00, 0.03]

The negative cells are material. At $k = 1$, FCFS-minus-Guard p99 differences are -208.61, -493.79 and -1,071.72 s for the 300, 600 and 1200 s promises; all simultaneous bands are wholly negative. At $k = 2$ all guard point estimates remain negative. Positive simultaneous lower bounds occur for SPJF-E at $k = 2$ –12, Guard(300) and Guard(600) at $k = 4$ –12, and Guard(1200) at $k = 3$ –12. At $k = 14$ the guard point improvements are about 0.37 s but their simultaneous bands cross zero; at $k = 20$ the point differences are zero. A correct per-job bound does not imply positive tail benefit across queueing capacities.

For Guard(600) at $k = 4$ the maximum excess is 515.84 s and harm is 339.06 s, although it closes 0.8000 [0.7620, 0.8388] of the reference gap. At $k = 20$ that ratio is undefined because the reference gap is not positive, while maximum excess and harm remain 124.38 and 37.37 s. On overlay 0, FCFS still has one positive wait at $k = 75$; at $k = 76$ every policy has zero wait. The latter is the exact maximum infinite-server occupancy, hence a zero-wait endpoint for this overlay, not a capacity recommendation for a deployment.

S6.2. An Executing Timed-Work Service

The physical selection rule chooses the complete calendar-aligned 300-second bin with the greatest offered capped work in development overlay 0, breaking ties by earliest time. Its 2,151 jobs carry 4,118.215007 s of requested work, 6.864 times the two-worker release-window capacity. Each policy starts empty and drains its own queue. Inter-arrival spacing, requested costs and scores are retained; workers hold their slots with timed sleeps. This tests queueing and the control rule, not the portability of submitted-program execution time. It also discards inherited full-trace backlog, so its percentile is not the capacity sweep’s percentile.

The nominal payload cap is 60 s; the measured-service premise $L = 61$ s was fixed before execution. Guard parameters are $B_0 = 30$ s, $\eta = 0.5$, $\gamma = 0$ and $B_{\max} = 356$ s, corresponding to the nominal 300-second ideal-model promise. A measured holding time above L would fail the premise, not be clipped or used to raise L retrospectively. None did. Actual enqueue, choice, dispatch, worker start/finish and completion receipt are recorded separately; service becomes known only at completion receipt.

All 6,453 observed-prefix decisions passed an independent choice audit. Physical and ideal measured-input Guard firing counts were 322 and 372. The small mean discrepancies do not establish job-level fidelity: the maximum absolute per-job error is 392.84 s for SPJF-E and 1,354.10 s for Guard. Guard’s physical p99 exceeds the separate physical FCFS run

Table S8. Physical timed-work runs. The ideal comparator uses each run’s own measured arrivals and holding costs; all entries are seconds.

Policy	Mean wait	p99 wait	Max absolute per-job ideal discrepancy
FCFS	1,337.05	1,763.30	0.49
SPJF-E	142.79	1,687.47	392.84
Guard(300)	480.39	1,783.65	1,354.10

by 20.34 s. Thus the physical window confirms an executing queue and rule conformance but does not show a guarded p99 gain. Actual arrivals and holding costs differ between physical runs; their comparison is descriptive. One selected window supplies no across-window confidence interval.

The guard’s theorem reference is shadow FCFS on that guard run’s measured arrivals and service, not the separately timed physical FCFS run. An observed-trace certificate augments the ideal bound by cumulative idle worker-time and dispatch-to-start delay. Its largest correction is 1.127 s; all 2,151 jobs pass, and the largest uncorrected excess over shadow FCFS is 222.21 s. This is an ex post certificate conditional on the measured trace and the 61 s service premise. It does not bound future operating-system stalls and does not restore an advance uncorrected real-time promise.

S6.3. Why Open-Loop Benefit Has No Known Bias Direction

The replay compares policies conditional on fixed offered work. In an exact single-server feedback construction, jobs of length 10 and 1 s arrive together, the long job first, and the short job’s user submits a second unit job 0.5 s after completion. Replaying the FCFS-induced arrivals gives SJF a mean-wait advantage of 3.00 s; letting each policy generate its own successor arrival changes the advantage to -0.167 s. The corresponding linearly interpolated p99 advantages are 8.82 and 0.47 s. The job cohort is identical, so this example does not depend on comparing different populations.

The opposite bias direction is possible too. An initial FCFS order with services 20, 1, 3 and 10 s, whose unit-job user returns after 2 s with a 0.1 s successor, gives open- and closed-loop mean advantages of 9.20 and 11.16 s and p99 advantages of 10.00 and 10.18 s. These exact recursions falsify a general lower-bound interpretation; they are not fitted behavioural models. The pathwise theorem still compares a guarded schedule with shadow FCFS on that schedule’s realised arrivals and services, not independently evolving closed-loop systems [6].

S6.4. LPC-EGEE: Recorded Queueing with Weak Counterfactual Validation

The public LPC-EGEE trace supplies recorded waits and concentrated execution costs together [7]. After outage exclusion and queue-limit capping, the longest 1% of 206,222 retained jobs carries 40.7% of work. On the chronological evaluation part the top share is 35.1%, which is the value used in the main cross-domain table. The physical pools stay disjoint. Effective capacities 83 and 56 are fitted only on the earlier part, with no test-driven refit. Service is the minimum of recorded run time and its documented queue limit; the common bound uses the largest pool limit, 259,200 s.

The fitted FCFS approximation underestimates evaluation-part mean waits by 24% and 39%, with hourly correlations of 0.44 and 0.40. The archive describes plain FCFS serial-job ordering during the logged period, but queue concurrency ceilings, polling and middleware are absent from the work-conserving replay. Recorded waits therefore cannot be equated with the theoretical FCFS counterfactual. This is weak simulator validation, not evidence that the original scheduler was an arbitrary non-FCFS policy.

In the conditional replay, SPJF-E reduces mean wait by 21.4% [13.2%, 38.2%], but its busy-hour p99 reduction of 4.2% [-11.2%, 46.0%] remains unresolved. Busy hours use frozen arrival-count thresholds, not deadlines. All six constant/age work-budget guards at the tested multiples of L fire zero times; a small fraction of the numerical bound being spent is therefore not evidence of effective protection. Histories use recorded completion times, and 2,906 evaluation arrivals (2.5%) have an outcome unavailable under replayed SPJF-E. This row is a recorded-feature conditional counterfactual, not a closed-loop deployment result.

S7. Additional theory: scope, accounting, and instance-dependent bounds

This section records results that delimit, rather than enlarge, the main-text claims. The first example rules out instance optimality for the uniform guard. The exact identity that follows is a useful job-level accounting decomposition; its core is standard sample-path workload balance, and we do not treat it as a new conservation law. The final bounds replace the global timeout by realised order statistics, but in their actual-size form they are retrospective certificates under completion-only observation.

S7.1. Comparison with the nearest guarantee disciplines

Table S9. Scope of the nearest per-job guarantees and wrapper precedent.

Method	Reference discipline	Arrival assumptions	as-	Preemption	Information needed	Server model	Guarantee
This work	Same-input FCFS	Arbitrary finite path	fi-	No	Ranks, completions, public opaque base proposal	k identical servers	Deterministic per-job additive waiting excess; wrapper around any base
PFCFS [8]	Flow time without later submissions	Online leases	re-	Gang pre-emption	Processor demand; service time need not be known; pre-emption penalty	Rigid jobs	λ -fairness; PFCFS is $(2 + 2\delta)$ -fair with competitive efficiency bounds
Conservative backfilling [9]	Submission-time reservation	Online missions	sub-	No	Requested time and resources	Rigid jobs	Later backfills do not delay any reserved start
PV-EASY [10, 11]	Priority reservation	Online missions; prediction error allowed	sub-	Kill and restart	Runtime estimates and resources	Rigid jobs	Zero lower-priority delay to the protected reservation
Nudge [12]	FCFS	Stationary Poisson	No		Size class at arrival	One server	Response-time stochastic dominance for its model; at most one swap per job
Load Guardrails [13]	Bal- Unconstrained dispatcher	Stochastic heavy-traffic regime		SRPT servers	Size class and patch state	Parallel server pools	Any dispatcher becomes mean-response-time optimal in heavy traffic

S7.2. No uniform instance-optimality ratio

Proposition 1 (An unbounded instance-optimality ratio). *At $k = 1$, let three batch jobs have rank-order service times $(L, 1, 1)$, with $L \geq 2$, and set $G = L$. The cap of the parameter rule of the main text (its Corollary on parameters from a promise, $B_{\max} = k(G - (3 - 2/k)L)$) is $B = 0$, so the guard is FCFS and has total waiting time $2L + 1$. The order $(1, 1, L)$ has total waiting time 3 and satisfies the per-job promise G . Consequently the ratio between the guard's total wait and that of a feasible policy is at least $(2L + 1)/3$, which is unbounded as $L \rightarrow \infty$.*

Proof. FCFS produces waits $0, L, L + 1$. Under the competing order the two short jobs wait 0 and 1, and the long job waits 2. The long job's excess is therefore 2 and both short jobs have nonpositive excess, so $L \geq 2$ makes the order G -feasible. The comparison allows a clairvoyant feasible policy; restricting the comparator to completion-only policies would be a different question. $\square$

S7.3. Idle-capacity accounting

Let t_0 be the first arrival time, let $b_Q(u)$ be the number of busy servers under policy Q at time u , and define the cumulative unused capacity

$$\Gamma_Q(t) = \int_{t_0}^t (k - b_Q(u)) \, du.$$

This includes idle capacity caused by having fewer than k jobs in the system; work conservation excludes only deliberate idling while a job waits.

Lemma 1 (Exact accounting with idle-capacity correction). *For every finite input, every non-preemptive work-conserving policy P , and every job i ,*

$$k(W_P[i] - W_{\text{FCFS}}[i]) = \text{In}_i - \text{Out}_i + \Gamma_P(a_i) - \Gamma_{\text{FCFS}}(a_i) - \rho_i^P + \rho_i^{\text{FCFS}}.$$

The identity was checked on 214,735 job-instances, including tied events, non-batch arrivals, and same-phase overtakers.

Proof. Let $A(t)$ be cumulative arrived work and $E_Q(t)$ cumulative executed work. Then

$$U_Q(t) = A(t) - E_Q(t) = A(t) - k(t - t_0) + \Gamma_Q(t).$$

At a_i , after simultaneous arrivals and before that phase's dispatches, work from jobs not ranked below i is identical in the two runs. Hence

$$R_i^P - R_i^{\text{FCFS}} = U_P(a_i) - U_{\text{FCFS}}(a_i) = \Gamma_P(a_i) - \Gamma_{\text{FCFS}}(a_i).$$

Substitution into the two busy-period equalities of the main text (the displayed equality $k W_P[i] = R_i^P + \text{In}_i - \text{Out}_i - \rho_i^P$ in the proof of the identity), one for P and one for FCFS, proves the formula. A same-phase overtaker enters both In_i and ρ_i^P with its full size and therefore cancels. $\square$

Writing $D_i = \Gamma_P(a_i) - \Gamma_{\text{FCFS}}(a_i)$, the arrival-frozen budget

$$b_i = \max\{0, k(G - L) - (k - 1)L - D_i\}$$

has the mathematical promise G in the standard range $G \geq (3 - 2/k)L$: combine $\text{In}_i < b_i + kL$ with Lemma 1 and $\rho_i^{\text{FCFS}} \leq (k - 1)L$. This is not a deployable rule under the paper's completion-only observation model. In general $\Gamma_{\text{FCFS}}(a_i)$ depends on when an FCFS-shadow job completes before its size has been revealed in the realised run; simulating the shadow does not supply that missing completion time.

S7.4. Instance-dependent order-statistic bounds

For a set of jobs S , let $\Lambda_r(S)$ be the sum of its largest at most r original service times, with missing terms replaced by zero. We abbreviate

$$\Lambda_{k-1}(t) = \Lambda_{k-1}(\{j : a_j \leq t\}), \quad \Lambda_{k-1}^{<i}(t) = \Lambda_{k-1}(\{j : j < i\}),$$

and

$$\Lambda_k^{>i}(t) = \Lambda_k(\{j : j > i, a_j \leq t\}).$$

Proposition 2 (Instance-dependent workload gap). *For any two non-preemptive work-conserving policies Q_1, Q_2 ,*

$$|U_{Q_1}(t) - U_{Q_2}(t)| \leq \Lambda_{k-1}(t) \quad \text{for every } t.$$

Proof. The workload difference is continuous across arrivals, whereas $\Lambda_{k-1}(t)$ can only increase. If $U_{Q_1}(t) > \Lambda_{k-1}(t)$, policy Q_1 must have at least k jobs present: otherwise their remaining work is no larger than the sum of their at most $k-1$ original sizes. All its servers are therefore busy, so the difference cannot increase. It cannot cross the nondecreasing barrier from below; exchange Q_1 and Q_2 for the other direction. $\square$

Proposition 3 (Instance-dependent excess accounting). *For every policy P and job i ,*

$$k \text{ excess}_P[i] \leq \text{In}_i - \text{Out}_i + \Lambda_{k-1}(a_i) + \Lambda_{k-1}^{<i}.$$

Proof. In Lemma 1, Proposition 2 bounds the idle-capacity difference, equivalently the workload difference at a_i , by $\Lambda_{k-1}(a_i)$. Drop $-\rho_i^P \leq 0$. The at most $k-1$ FCFS jobs still running when i starts all have rank below i , so $\rho_i^{\text{FCFS}} \leq \Lambda_{k-1}^{<i}$. $\square$

Proposition 4 (Instance-dependent overtaking under the guard). *If job i has an overtaker and τ is the dispatch time of its last overtaker, then*

$$\text{In}_i < \text{budget}(i, \tau) + \Lambda_k^{>i}(s_i^P).$$

If the budget is nondecreasing in time, the first term may be replaced by $\text{budget}(i, s_i^P)$; with only a uniform cap it may be replaced by $B_{\max}$.

Proof. Completed overtaking work before the last overtaker is dispatched is strictly below $\text{budget}(i, \tau)$. At τ , at most k dispatched overtakers remain unfinished: the newly dispatched job and at most one on each other server. Their total original size is at most $\Lambda_k^{>i}(s_i^P)$. $\square$

Proposition 5 (Combined instance-dependent guard bound). *Under a cap $B_{\max}$, every job satisfies*

$$k \text{ excess}_P[i] \leq B_{\max} + \Lambda_k^{>i}(s_i^P) + \Lambda_{k-1}(a_i) + \Lambda_{k-1}^{<i},$$

and the inequality is strict when i has an overtaker. When it has none, the sharper weak bound omits the first two terms.

Proof. Combine Propositions 3 and 4, use $\text{Out}_i \geq 0$, and apply the cap. If no job overtakes i , then $\text{In}_i = 0$ and Proposition 3 gives the stated sharper form. $\square$

These Λ quantities use actual sizes, including sizes of unfinished jobs, and hence are offline instance certificates in the completion-only model. Replacing each unknown size by a public class-specific upper bound gives a deployable but generally looser inequality. The homogeneous substitution recovers the main text's aggregate worst-case constant; it does not prove that each order-statistic term is separately minimal.

As an illustration, consider independently sampled truncated-Pareto sizes with $\alpha = 1.5$, $x_{\min} = 1$, hard cap 1000, sample size $N = 200$, and 5,000 samples drawn with seed 73621. The first ratio below sets L after the fact to the maximum of each finite sample; the second uses the fixed hard cap as the denominator.

k	$\mathbb{E}[\Lambda_{k-1}/((k-1)L_{\text{sample}})]$	$\mathbb{E}[\Lambda_{k-1}/((k-1)L_{\text{cap}})]$
4	0.6825	0.0444
8	0.4648	0.0263
16	0.3139	0.0161

Thus the larger ratios use $L = L_{\text{sample}} = \max_j C_j$, not the published timeout $L_{\text{cap}} = 1000$.

References

1. Universidade Federal do Amazonas, Instituto de Computação. CodeBench Dataset. <https://codebench.icomp.ufam.edu.br/dataset/>. Accessed 2026-09-19. 498
2. Microsoft Azure. Azure Functions Invocation Trace 2021. <https://github.com/Azure/AzurePublicDataset/blob/master/AzureFunctionsInvocationTrace2021.md>, 2021. 499
3. Parallel Workloads Archive. The Intel Netbatch Logs. https://www.cs.huji.ac.il/labs/parallel/workload/1_intel_netbatch/, 2012. 500
4. Mozilla. Taskcluster Documentation: Queue Service Reference. <https://docs.taskcluster.net/docs/reference/platform/queue>. Accessed 2026-09-19. 501
5. Mozilla. Treeherder REST API Documentation. <https://treeherder.mozilla.org/docs/>. Accessed 2026-09-19. 502
6. Shmueli, E.; Feitelson, D.G. On Simulation and Design of Parallel-Systems Schedulers: Are We Doing the Right Thing? *IEEE Transactions on Parallel and Distributed Systems* **2009**, *20*, 983–996. <https://doi.org/10.1109/TPDS.2008.152>. 503
7. Tsafir, D. The LPC Log. https://www.cs.huji.ac.il/labs/parallel/workload/1_lpc/. Parallel Workloads Archive; data provided by Emmanuel Medernach; accessed 2026-09-23. 504
8. Schwiegelshohn, U.; Yahyapour, R. Fairness in Parallel Job Scheduling. *Journal of Scheduling* **2000**, *3*, 297–320. [https://doi.org/10.1002/1099-1425\(200009/10\)3:5<297::AID-JOS50>3.0.CO;2-D](https://doi.org/10.1002/1099-1425(200009/10)3:5<297::AID-JOS50>3.0.CO;2-D). 505
9. Lindsay, A.M.; Galloway-Carson, M.; Johnson, C.R.; Bunde, D.P.; Leung, V.J. Backfilling with guarantees made as jobs arrive. *Concurrency and Computation: Practice and Experience* **2013**, *25*, 513–523. <https://doi.org/10.1002/cpe.2860>. 506
10. Yuan, Y.; Yang, G.; Wu, Y.; Zheng, W. PV-EASY: A Strict Fairness Guaranteed and Prediction Enabled Scheduler in Parallel Job Scheduling. In Proceedings of the Proceedings of the 19th ACM International Symposium on High Performance Distributed Computing (HPDC '10), 2010, pp. 240–251. <https://doi.org/10.1145/1851476.1851505>. 507
11. Yuan, Y.; Wu, Y.; Zheng, W.; Li, K. Guarantee Strict Fairness and Utilize Prediction Better in Parallel Job Scheduling. *IEEE Transactions on Parallel and Distributed Systems* **2014**, *25*, 971–981. <https://doi.org/10.1109/TPDS.2013.88>. 508
12. Grosz, I.; Yang, K.; Scully, Z.; Harchol-Balter, M. Nudge: Stochastically Improving upon FCFS. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* **2021**, *5*, 1–29. Preprint arXiv:2106.01492, <https://doi.org/10.1145/3460088>. 509
13. Grosz, I.; Scully, Z.; Harchol-Balter, M. Load Balancing Guardrails: Keeping Your Heavy Traffic on the Road to Low Response Times. arXiv preprint arXiv:1905.03439, 2019, [arXiv:cs.PF/1905.03439]. <https://doi.org/10.48550/arXiv.1905.03439>. 510